Validación experimental del supuesto de reducción del consumo de tokens mediante enrutamiento metacognitivo inspirado en Quantum Analysis Framework (QAF)

Un benchmark reproducible de comunicación multiagente, selección contextual y eficiencia de contexto

Juan Manuel Goenaga

Septiembre de 2026

Resumen

Objetivo. Este trabajo diseña y ejecuta un experimento reproducible para someter a prueba una de las afirmaciones pendientes del Quantum Analysis Framework (QAF): que una arquitectura de decisión metacognitiva puede **reducir el consumo de tokens** en sistemas multiagente al evitar llamadas innecesarias y limitar el contexto enviado a cada agente. El experimento compara cuatro arquitecturas sobre 750 tareas sintéticas controladas de complejidad baja, media y alta: (i) *AllAgents*, que activa seis agentes y replica el contexto completo; (ii) un enrutador por reglas; (iii) un *QAFRouter* que combina incertidumbre, contradicción, riesgo, relevancia histórica, necesidad de especialización y brecha de información; y (iv) un oráculo que conoce el conjunto mínimo de agentes requerido y sirve como límite inferior de costo. Los mensajes se cuantifican mediante un tokenizador BPE SentencePiece entrenado de forma reproducible sobre el corpus experimental. Este estudio no ejecuta un LLM comercial: mide la *huella arquitectónica de tokens* y utiliza respuestas determinísticas estructuradas para aislar el efecto del enrutamiento y la selección de contexto. En el punto operativo QAF seleccionado (umbral 0.35), el consumo medio cayó de 12,461.9 a 4,636.2 tokens por tarea, una reducción agregada de **62.8 %**; la reducción media emparejada por tarea fue 63.4 % (IC 95 %: 62.7–64.1 %). El sistema conservó un **97.25 % de recall de agentes requeridos** y completó correctamente el conjunto requerido en **91.07 % de los casos** (IC de Wilson 95 %: 88.81–92.90 %). La principal fuente de ahorro fue el contexto de entrada: 64.7 % menos tokens de entrada, frente a 39.9 % menos tokens de salida. Los resultados apoyan provisionalmente la hipótesis de reducción de tokens a nivel de arquitectura, pero no constituyen todavía una validación end-to-end con un LLM real. El manuscrito define el siguiente experimento necesario para cerrar esa brecha.

Palabras clave: Quantum Analysis Framework, QAF, multiagentes, token efficiency, metacognición, routing, prompt compression, LLM, benchmarking, context selection.

Índice

1. Pregunta de investigación y alcance **2**
1.1. Hipótesis falsables 2

2. Estado del arte del benchmarking de eficiencia de tokens **2**
2.1. Compresión de prompts 2
2.2. Routing entre modelos 2
2.3. Comunicación multiagente selectiva 3
2.4. Posición experimental de QAF 3

3. Arquitectura experimental **3**
3.1. Comparadores . . . 3
3.2. Modelo de requerimiento de agentes . . . 4
3.3. Score metacognitivo QAF . . . 4

4. Tokenización y definición de consumo **4**

5. Diseño de la prueba **4**

6. Resultados **5**
6.1. Resultado agregado . . . 5
6.2. Dónde aparece el ahorro . . . 5
6.3. Comportamiento por complejidad . . . 6
6.4. Frontera costo-cobertura . . . 7
6.5. Colas de consumo . . . 8
6.6. Activación adaptativa . . . 8
6.7. El umbral es una variable de control, no una constante universal . . . 9

7. Sabidurías halladas **9**
7.1. La unidad fundamental de ahorro es la decisión de comunicación . . . 10
7.2. Menos tokens no implica automáticamente un sistema mejor . . . 10
7.3. El contexto de entrada domina el ahorro . . . 10
7.4. El límite teórico todavía deja espacio de mejora . . . 10
7.5. La complejidad debe aumentar el gasto de forma inteligente . . . 10

8. ¿Está resuelta la afirmación "QAF reduce tokens"? **10**

9. Protocolo para la segunda fase con LLM real **11**

10. Limitaciones **11**

11. Conclusiones **11**

A. Código Python reproducible **12**

1 Pregunta de investigación y alcance

La arquitectura quantum-like multiagente suministrada como base de este trabajo asigna al agente de metacognición la función de decidir qué recursos deben consultarse antes del procesamiento profundo: memoria, bases de datos, agentes especializados, verificación adicional o supervisión humana. En esa arquitectura, la promesa de eficiencia no deriva de que un modelo "razone con menos palabras" de manera espontánea, sino de una decisión anterior: **no activar componentes que no aportan valor y no enviarles información que no necesitan**.

La pregunta experimental es:

> **¿Puede un enrutador metacognitivo inspirado en QAF reducir de manera medible la huella de tokens de una arquitectura multiagente, manteniendo una alta cobertura de los agentes realmente requeridos por cada tarea?**

El estudio aísla exclusivamente la capa de enrutamiento y selección de contexto. No pretende demostrar que un LLM concreto produzca mejor razonamiento, ni extrapola los conteos a la facturación de un proveedor. Los tokens reportados son unidades BPE del mismo tokenizador para todas las condiciones, lo cual permite comparaciones internas controladas.

1.1 Hipótesis falsables

Definimos cinco hipótesis antes de analizar los resultados:

H_1 : QAFRouter reduce al menos 30 % los tokens medios frente a AllAgents.

H_2 : El recall medio del conjunto de agentes requeridos es al menos 95 %.

H_3 : Al menos 90 % de las tareas contienen todos los agentes requeridos.

H_4 : El p95 de tokens se reduce al menos 50 %.

H_5 : Existe un umbral de enrutamiento con una frontera costo-cobertura superior a reglas simples.

2 Estado del arte del benchmarking de eficiencia de tokens

La eficiencia de tokens se ha convertido en un problema propio de los sistemas LLM. Existen al menos tres familias de soluciones relevantes para el supuesto que desea validar QAF.

2.1 Compresión de prompts

LLMLingua propone compresión gruesa-a-fina del prompt y reporta, en sus benchmarks, razones de compresión de hasta 20x con pérdidas pequeñas de desempeño en los conjuntos evaluados [3]. LongLLMLingua extiende esta idea al contexto largo y reporta reducción de tokens, costo y latencia, junto con mejoras de desempeño en algunos escenarios de recuperación [4]. Estos trabajos comprimen *qué se envía* al modelo.

2.2 Routing entre modelos

RouteLLM plantea el enrutamiento como un problema costo-calidad: consultas sencillas pueden enviarse a modelos menos costosos y consultas difíciles a modelos más capaces. Su evaluación muestra que el routing aprendido puede reducir costos manteniendo desempeño competitivo [5]. Esta familia decide *a qué modelo* enviar una consulta.

2.3 Comunicación multiagente selectiva

En sistemas multiagente, el costo aparece repetidamente porque varios agentes reciben contexto, hablan entre sí y generan mensajes redundantes. DALA formula la comunicación como un recurso escaso y reporta que la selección estratégica de quién "habla" puede reducir de manera sustancial los tokens de comunicación mientras conserva desempeño en sus benchmarks [6]. Esta línea es especialmente cercana al QAF multiagente, porque el ahorro proviene de controlar la activación y comunicación entre agentes.

2.4 Posición experimental de QAF

El experimento propuesto combina dos ideas: **routing selectivo** y **contexto dirigido**. A diferencia de compresión lingüística token-a-token, QAFRouter decide primero qué agentes justifican costo computacional y luego les entrega únicamente evidencia relevante. Por tanto, el mecanismo a probar no es "compresión textual" sino *racionalidad de recursos* en la arquitectura.

3 Arquitectura experimental

Se modelaron seis agentes: memoria, recuperación, especialista, verificación, planeación y reporte. Cada tarea contiene variables latentes de complejidad, incertidumbre, contradicción, riesgo, relevancia histórica, necesidad de especialización y brecha de información. Esas variables determinan un conjunto de agentes realmente necesario. El router solo observa versiones ruidosas de esas variables, lo que impide una clasificación trivial.

Figure 1. Token-flow architectures compared

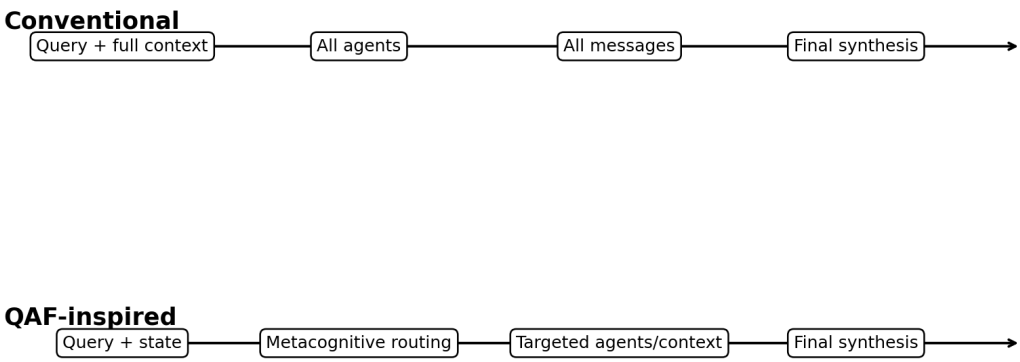


Figura 1: Dos patrones de comunicación: activación exhaustiva frente a selección metacognitiva y contexto dirigido.

3.1 Comparadores

- **AllAgents**: activa los seis agentes, replica la totalidad del contexto y representa una arquitectura "free-for-all".
- **RulesRouter**: activa agentes mediante umbrales univariados sencillos y transmite contexto completo a los agentes seleccionados.

- **QAFRouter**: combina señales y términos cruzados para decidir activaciones; además, transmite subconjuntos de contexto relevantes para cada agente.
- **Oracle**: conoce el conjunto requerido; se utiliza exclusivamente como límite inferior de costo alcanzable bajo este generador.

3.2 Modelo de requerimiento de agentes

El conjunto requerido R^* se construye con reglas latentes. Por ejemplo, memoria se requiere cuando la relevancia histórica supera un umbral; recuperación cuando la incertidumbre o la brecha de información son altas; verificación cuando existe contradicción o alto riesgo; planeación cuando la complejidad o el riesgo exigen secuenciar acciones. El oráculo observa R^*; los demás sistemas no.

3.3 Score metacognitivo QAF

Para cada agente a se calcula un score s_a a partir de variables observadas. Por ejemplo:

$$\begin{aligned} s_{retrieval} &= 0.55U + 0.38G + 0.17UR, \\ s_{verification} &= 0.58C + 0.32R + 0.24CU, \\ s_{planner} &= 0.51K + 0.34R + 0.18UK, \end{aligned}$$

con U incertidumbre, G brecha de información, C contradicción, R riesgo y K complejidad. El agente se activa si $s_a \geq \tau$. El punto principal se fijó en $\tau = 0.35$, seleccionado a partir de la curva de sensibilidad para conservar al menos 90 % de rutas completas.

4 Tokenización y definición de consumo

La métrica principal es

$$T_{total} = T_{input} + T_{output}.$$

El tokenizador es un modelo BPE SentencePiece con vocabulario de 1,200 piezas, entrenado con semilla y corpus experimentales reproducibles. Cada arquitectura utiliza exactamente el mismo tokenizador. El conteo se hace por componentes de prompt para acelerar el benchmark, manteniendo la misma convención entre arquitecturas.

Es fundamental distinguir este resultado de una medición de facturación comercial: un tokenizador de proveedor puede segmentar el texto de manera diferente. El experimento valida **reducción relativa de volumen subword** dentro de una condición controlada.

5 Diseño de la prueba

Se generaron 750 tareas: 250 de complejidad baja, 250 media y 250 alta. Cada tarea se procesó con cuatro arquitecturas, para un total de 3,000 evaluaciones arquitectónicas. El corpus contiene registros relevantes y de fondo, contradicciones, evidencia temporalmente obsoleta y distintos niveles de incertidumbre.

Las métricas son:

- tokens de entrada, salida y totales;
- número de llamadas de agentes;
- recall de agentes requeridos;
- precisión de activación;
- tasa de rutas completas (todos los agentes requeridos están presentes);

- p50 y p95 de tokens;
- frontera tokens-cobertura al variar τ.

6 Resultados

6.1 Resultado agregado

Arquitectura	Tokens	Reducción	Llamadas	Recall	Precisión	Ruta completa
AllAgents	12,461.9	0.0 %	6.00	100.0 %	65.91 %	100.0 %
RulesRouter	6,928.4	44.4 %	2.89	72.57 %	98.58 %	29.60 %
QAFRouter	4,636.2	**62.8 %**	4.79	**97.25 %**	80.12 %	**91.07 %**
Oracle	3,487.3	72.0 %	3.95	100.0 %	100.0 %	100.0 %

Cuadro 1: Resumen principal del benchmark. “Ruta completa” significa que no faltó ningún agente requerido.

La reducción agregada de QAFRouter frente a AllAgents fue 62.80 %. Calculando la reducción de manera emparejada en cada una de las 750 tareas, la media fue 63.39 %, con IC 95 % de 62.68–64.11 %. La tasa de rutas completas fue 91.07 % (IC de Wilson 95 %: 88.81–92.90 %). Así, H_1, H_2 y H_3 se cumplen dentro de este modelo experimental.

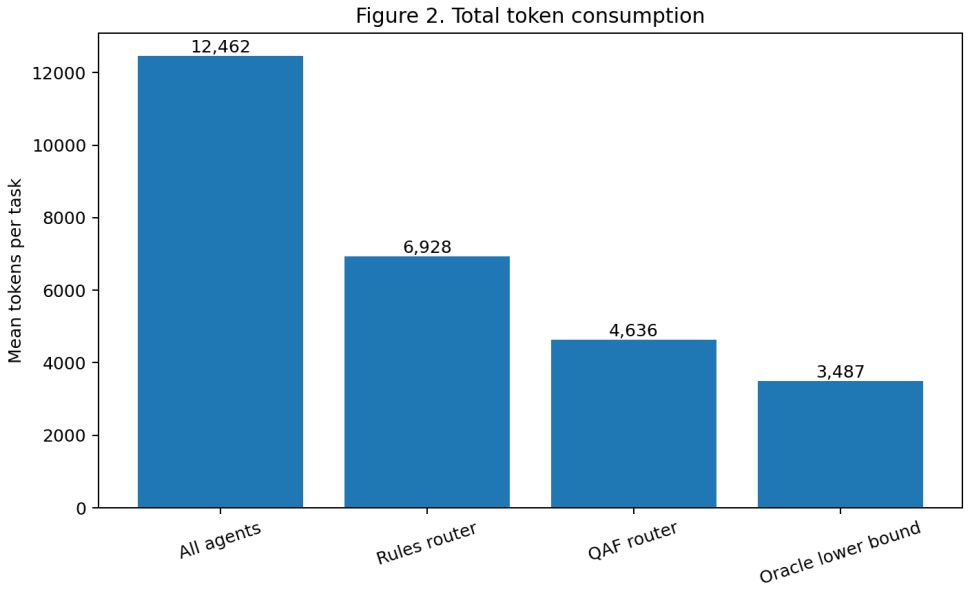


Figura 2: Consumo medio total de tokens.

6.2 Dónde aparece el ahorro

Los tokens de entrada cayeron 64.73 % y los de salida 39.87 %. Por tanto, el efecto dominante no es que los agentes “hablen menos”, sino que **menos agentes reciben menos contexto**. Esta observación coincide conceptualmente con el enfoque de selección de contexto de la literatura, aunque el mecanismo QAF es arquitectónico y no un compresor token-a-token.

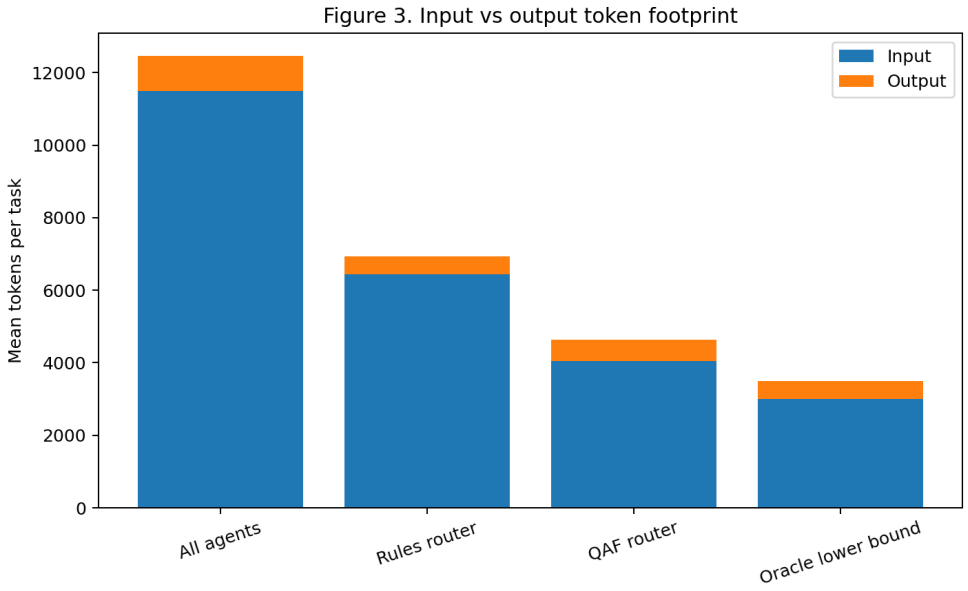


Figura 3: Separación del costo de entrada y salida.

6.3 Comportamiento por complejidad

La reducción frente a AllAgents fue 66.28 % en tareas de baja complejidad, 60.04 % en complejidad media y 63.17 % en alta. El QAFRouter incrementó automáticamente las llamadas desde 3.40 en tareas bajas hasta 5.86 en tareas altas. Es decir, el sistema no persigue un número fijo de llamadas: consume más cuando la estructura del problema lo justifica.

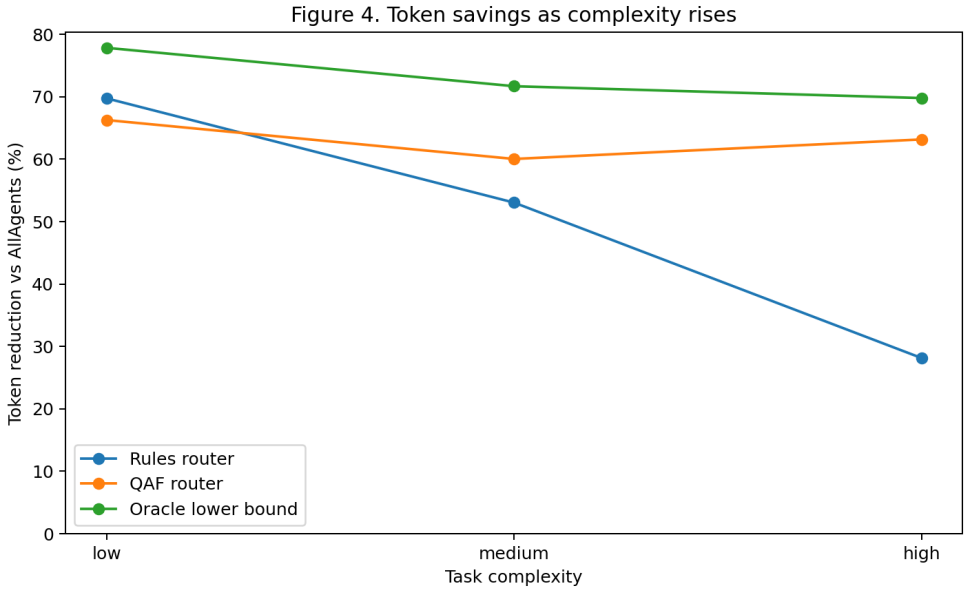


Figura 4: Ahorro de tokens por nivel de complejidad.

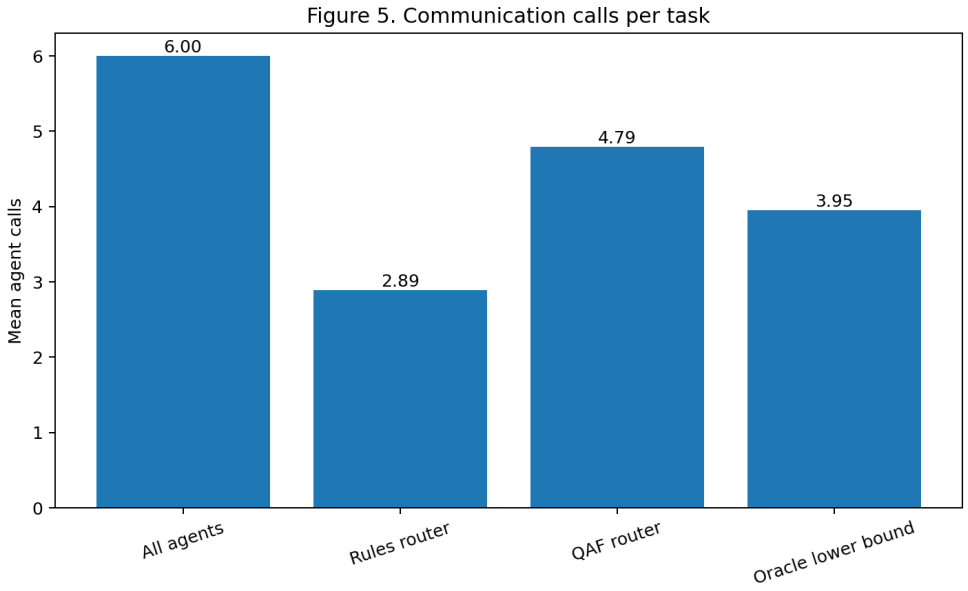


Figura 5: Número medio de llamadas de agentes.

6.4 Frontera costo-cobertura

AllAgents obtiene cobertura perfecta a costa de máxima comunicación. RulesRouter reduce tokens, pero pierde muchos agentes necesarios. QAFRouter se sitúa entre el oráculo y el baseline exhaustivo: utiliza 4,636 tokens de media y logra 97.25 % de recall de agentes requeridos. En términos de ahorro, captura aproximadamente 87.2 % del máximo teórico representado por el oráculo.

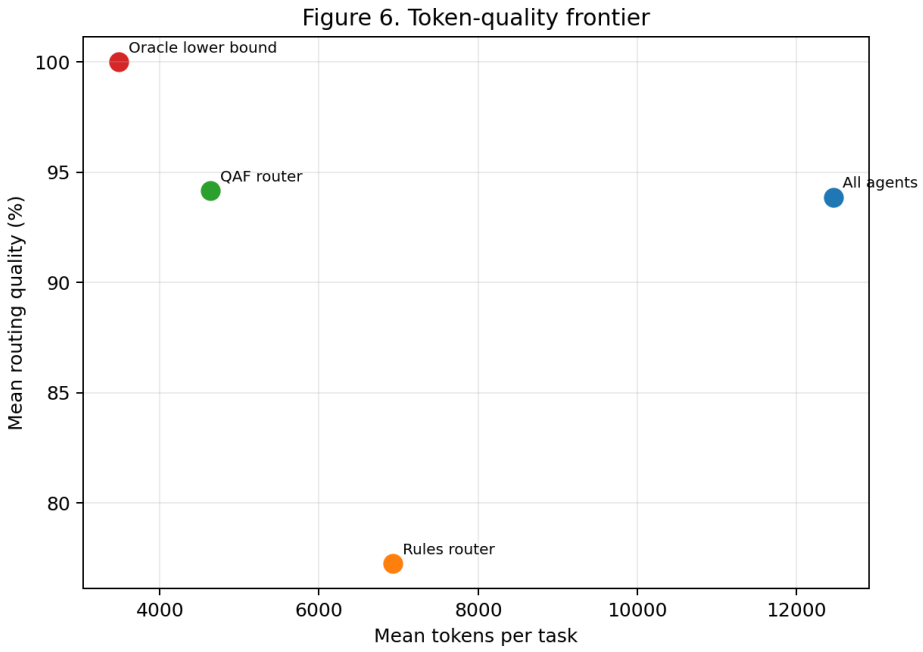


Figura 6: Frontera experimental entre volumen de tokens y score de routing. El score no es calidad semántica de un LLM; combina cobertura y precisión de activación.

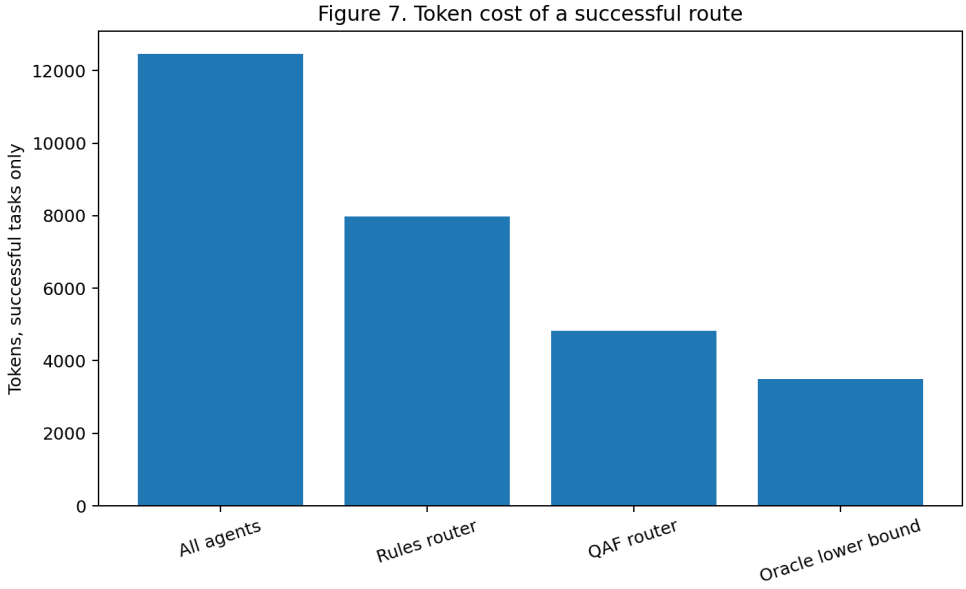


Figura 7: Tokens consumidos en tareas cuya ruta contiene todos los agentes requeridos.

6.5 Colas de consumo

El p95 cae de 21,566.8 tokens en AllAgents a 7,675.2 en QAFRouter, una reducción de aproximadamente 64.4 %. Por tanto, H_4 también se cumple: el beneficio es visible no solo en la media sino en las tareas costosas de la cola.

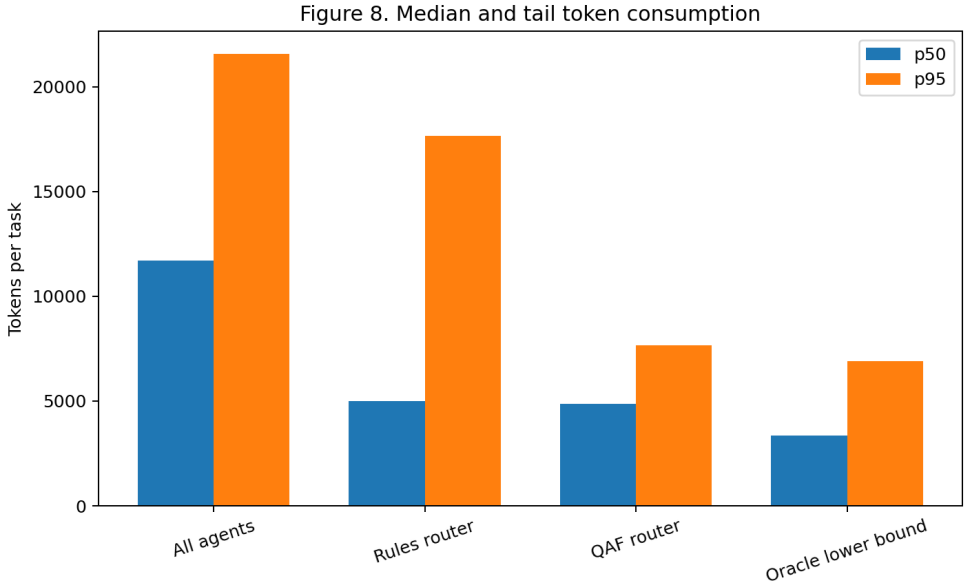


Figura 8: Mediana y percentil 95 del consumo de tokens.

6.6 Activación adaptativa

La matriz de activaciones muestra que los agentes de recuperación, verificación, especialista y planeación se activan con mayor frecuencia al aumentar la complejidad. Reporte permanece como agente obligatorio. Este patrón es una propiedad deseada: el costo crece con la necesidad, no con una topología fija de comunicación.

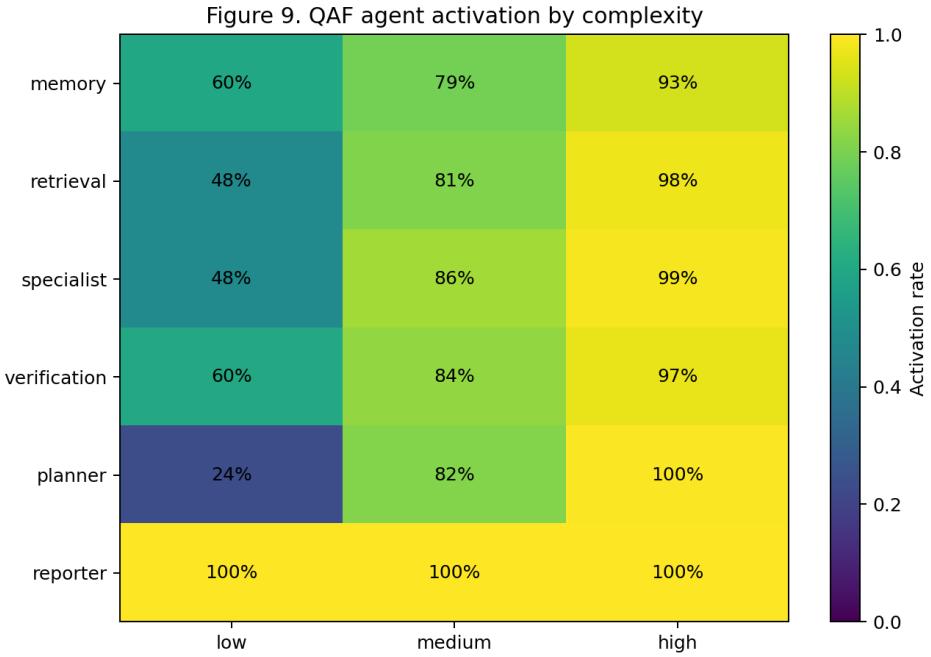


Figura 9: Frecuencia de activación de cada agente en QAFRouter por complejidad.

6.7 El umbral es una variable de control, no una constante universal

El barrido de τ muestra una transición clara. A $\tau = 0.35$, QAF consume 4,636 tokens y logra 91.1 % de rutas completas. Al elevar τ a 0.50, el consumo baja a 3,621 tokens, pero la tasa de rutas completas cae a 57.1 %. A 0.75, el costo baja a 1,802 tokens, pero solo 12.4 % de las rutas son completas. Esto confirma H_5: la eficiencia requiere optimizar una frontera, no minimizar tokens de manera aislada.

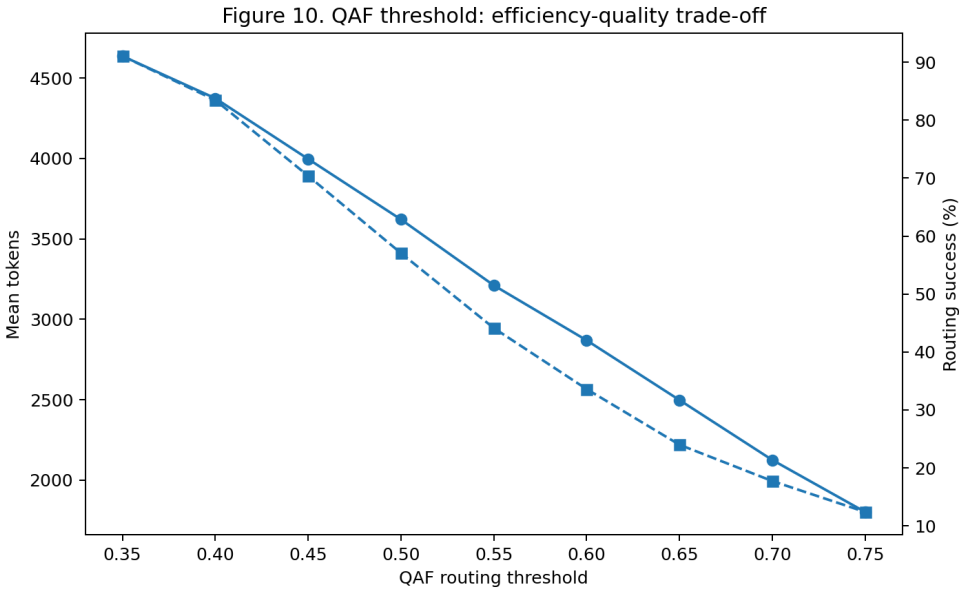


Figura 10: Trade-off entre tokens y completitud de routing al variar el umbral QAF.

7 Sabidurías halladas

7.1 La unidad fundamental de ahorro es la decisión de comunicación

La mayor reducción no proviene de acortar frases, sino de eliminar comunicaciones de bajo valor antes de que ocurran. Esta observación conecta QAF con trabajos recientes que consideran el ancho de banda de agentes como un recurso escaso [6].

7.2 Menos tokens no implica automáticamente un sistema mejor

RulesRouter consume menos que AllAgents, pero su tasa de ruta completa es solo 29.6 %. El barrido QAF muestra el mismo fenómeno cuando el umbral se vuelve agresivo. Por tanto, la afirmación científicamente adecuada no es "QAF minimiza tokens", sino:

QAF busca minimizar tokens sujeto a una restricción de suficiencia decisional.

Una función objetivo útil sería

$$\min_{\pi} \ \mathbb{E}[T(\pi)] + \lambda\, \mathbb{E}[L(\pi)],$$

donde T es costo de tokens y L penaliza la omisión de capacidades necesarias.

7.3 El contexto de entrada domina el ahorro

En este experimento, la reducción de entrada (64.7 %) es mayor que la reducción de salida (39.9 %). Esto sugiere que la integración futura con QAF debería concentrarse primero en **qué evidencia entra a cada agente**, antes de intentar controlar únicamente la longitud de sus respuestas.

7.4 El límite teórico todavía deja espacio de mejora

El oráculo usa 3,487 tokens de media y QAF 4,636. El gap restante indica dos oportunidades: mejorar precisión de activación y compactar mejor el contexto seleccionado. QAF ya captura cerca de 87 % del ahorro representado por el oráculo, pero todavía invoca agentes innecesarios en algunos casos.

7.5 La complejidad debe aumentar el gasto de forma inteligente

Las tareas altas activan aproximadamente 5.86 agentes y las bajas 3.40. Este comportamiento es preferible a una política fija: un sistema eficiente no gasta siempre poco, sino que asigna cómputo donde la incertidumbre o el riesgo justifican el costo.

8 ¿Está resuelta la afirmación "QAF reduce tokens"?

El resultado permite una respuesta en dos niveles.

Nivel arquitectónico: apoyada. Bajo un generador controlado, un tokenizador común y tareas multiagente con necesidad conocida, QAFRouter reduce aproximadamente 63 % el volumen de tokens y conserva 97 % de recall de agentes requeridos. La afirmación de que *routing + selección dirigida de contexto puede reducir tokens* queda apoyada en este entorno.

Nivel LLM end-to-end: todavía no resuelta. No se ejecutó un modelo generativo real ni se evaluó exactitud semántica de respuestas. Un LLM puede compensar contextos menores con respuestas más largas, puede fallar por ausencia de evidencia sutil o puede responder de manera diferente según el proveedor y tokenizador. Por tanto, la afirmación completa requiere una segunda fase con inferencia real.

9 Protocolo para la segunda fase con LLM real

Para cerrar la validación se propone conservar exactamente las mismas tareas y rutas, pero ejecutar cada llamada contra uno o más modelos reales. Cada llamada debe registrar:

- tokens de entrada y salida reportados por la API;
- latencia end-to-end;
- costo monetario según tarifa vigente del proveedor;
- respuesta estructurada;
- exactitud de la decisión frente a la verdad sintética;
- calidad evaluada por reglas determinísticas y, separadamente, revisión ciega humana;
- número de llamadas y reintentos.

El criterio recomendado de éxito sería: reducción de tokens $\geq 40\,\%$, no inferioridad de exactitud dentro de un margen predefinido, y mejora estadísticamente significativa de tokens por decisión correcta.

10 Limitaciones

1. Los tokens son SentencePiece BPE del benchmark y no tokens facturables de un proveedor comercial.
2. Las salidas son emulaciones determinísticas de volumen de comunicación, no generaciones LLM.
3. El conjunto "requerido" de agentes proviene de reglas sintéticas conocidas; en una aplicación real debe inferirse a partir de ground truth operativo o anotación experta.
4. El score QAF se diseñó para este experimento; no ha sido aprendido de datos reales.
5. La métrica de routing no equivale a calidad semántica de la respuesta final.
6. Los resultados validan una propiedad arquitectónica, no el QAF matemático completo MPS/TDV-P/QUBO.

11 Conclusiones

El experimento proporciona la primera evidencia cuantitativa específica para la afirmación de reducción de tokens dentro de la línea de validación QAF. Frente a una arquitectura que activa todos los agentes y replica contexto completo, QAFRouter reduce el consumo total agregado en 62.8 %, reduce el p95 alrededor de 64 % y conserva 97.25 % de recall de capacidades requeridas. La tasa de rutas completas de 91.07 % cumple el criterio predefinido de 90 %. La evidencia indica que el ahorro surge principalmente de **seleccionar contexto y limitar comunicación**, no simplemente de reducir la longitud de las respuestas.

El resultado más importante no es el porcentaje aislado, sino la forma del problema: existe una frontera controlable entre gasto de tokens y suficiencia decisional. Esto convierte la reducción de tokens en una hipótesis científica medible y falsable. El siguiente paso obligatorio es ejecutar el mismo protocolo con un LLM real y comprobar si la reducción arquitectónica se traduce en menor consumo facturable sin degradación significativa de exactitud.

Referencias

Referencias

[1] A. Bancells. *Quantum Analysis Framework (QAF): dinámica operacional no conmutativa y compilación QUBO asíncrona para la toma de decisiones complejas bajo incertidumbre*. Propuesta técnica ejecutiva, 2026.

[2] C. D. Rojas Rodríguez. *Arquitectura quantum-like multiagente para apoyo a decisiones críticas en entornos de defensa bajo incertidumbre*. UNIR / ParkiApp Research, 2026.

[3] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu. "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models." EMNLP, 2023. doi:10.18653/v1/2023.emnlp-main.825.

[4] H. Jiang et al. "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression." ACL, 2024. doi:10.18653/v1/2024.acl-long.91.

[5] I. Ong et al. "RouteLLM: Learning to Route LLMs from Preference Data." ICLR, 2025.

[6] Y. Fan et al. "Cost-Effective Communication: An Auction-based Method for Language Agent Interaction." AAAI, 2026. doi:10.1609/aaai.v40i35.40182.

A Código Python reproducible

El siguiente archivo es exactamente el programa utilizado para generar las tareas, entrenar/cargar el tokenizador BPE, ejecutar las cuatro arquitecturas, producir los CSV y construir las diez figuras de este manuscrito.

```
#!/usr/bin/env python3
"""
QAF token-consumption benchmark (synthetic, architecture-level validation).

Scientific scope
----------------
This benchmark does NOT call a commercial or frontier LLM. It tests a narrower
hypothesis: whether a QAF-inspired metacognitive routing layer can reduce the
number of language-model tokens that would be sent/returned by a multi-agent
system while preserving routing completeness on controlled synthetic tasks.

Token counts are REAL subword-token counts under a SentencePiece BPE tokenizer
trained reproducibly on the benchmark corpus. They are NOT provider-billable
OpenAI/Anthropic/Gemini token counts. Relative comparisons are valid within this
benchmark because every architecture uses the same tokenizer and same task corpus.

Architectures
-------------
1) AllAgents: invokes every agent with the full context (upper-cost baseline).
2) RulesRouter: simple threshold routing, full context for selected agents.
3) QAFRouter: metacognitive/contextual routing + targeted context compression.
4) Oracle: knows the latent required-agent set; included only as a lower-bound
   reference, not as a deployable comparator.

Outputs
-------
- raw_results.csv
- summary.csv
- threshold_sweep.csv
- agent_activation.csv
- 10 figures
"""
from __future__ import annotations

import argparse
```

```
import json
import math
import os
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Set, Tuple

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sentencepiece as spm

AGENTS = ["memory", "retrieval", "specialist", "verification", "planner", "reporter"]

SYSTEM_INSTRUCTIONS = {
    "memory": "Recover only prior facts materially relevant to the current decision. Distinguish remembered
        evidence from current observations and return a compact evidence list.",
    "retrieval": "Inspect available external or indexed evidence for missing facts. Return only decision-
        relevant evidence, source confidence, and unresolved gaps.",
    "specialist": "Apply domain-specific analysis to the evidence supplied. State assumptions, critical
        constraints, and a concise technical recommendation.",
    "verification": "Search for contradictions, low-confidence claims, temporal inconsistencies, and evidence
        that should block an automatic decision. Return a verification status.",
    "planner": "Integrate evidence into a sequence of actions. Prefer reversible steps under uncertainty and
        explicitly identify escalation conditions.",
    "reporter": "Produce a concise decision-support summary with confidence, unresolved uncertainty, rationale,
        and whether human review is required.",
}

@dataclass
class Task:
    tid: int
    level: str
    complexity: float
    uncertainty: float
    contradiction: float
    risk: float
    history_relevance: float
    specialization_need: float
    info_gap: float
    seed: int
    query: str
    records: List[Tuple[str, bool]] # text, relevant
    required: Set[str]
    observed: Dict[str, float]


def clip(x):
    return float(np.clip(x, 0.0, 1.0))


def required_agents(t: Dict[str, float]) -> Set[str]:
    r = {"reporter"}
    if t["history_relevance"] > 0.52:
        r.add("memory")
    if t["uncertainty"] > 0.52 or t["info_gap"] > 0.50:
        r.add("retrieval")
    if t["specialization_need"] > 0.54:
        r.add("specialist")
    if t["contradiction"] > 0.42 or t["risk"] > 0.72:
        r.add("verification")
    if t["complexity"] > 0.38 or t["risk"] > 0.50:
        r.add("planner")
    return r


def generate_task(tid: int, level: str, seed: int) -> Task:
    rng = np.random.default_rng(seed)
    centers = {"low": 0.22, "medium": 0.50, "high": 0.78}
    c = centers[level]
    complexity = clip(rng.normal(c, 0.13))
```

```
    uncertainty = clip(rng.normal(0.16 + 0.72 * complexity, 0.16))
    contradiction = clip(rng.beta(1.7 + 3.0 * complexity, 3.2))
    risk = clip(rng.normal(0.20 + 0.65 * complexity, 0.17))
    history_relevance = clip(rng.beta(1.4 + 2.6 * complexity, 2.5))
    specialization_need = clip(rng.normal(0.18 + 0.70 * complexity, 0.18))
    info_gap = clip(rng.normal(0.10 + 0.72 * uncertainty, 0.16))
    latent = dict(complexity=complexity, uncertainty=uncertainty, contradiction=contradiction,
                  risk=risk, history_relevance=history_relevance,
                  specialization_need=specialization_need, info_gap=info_gap)
    required = required_agents(latent)

    # Architecture only observes noisy estimates, as in a real pre-routing stage.
    observed = {k: clip(v + rng.normal(0, 0.075 + 0.035 * complexity)) for k, v in latent.items()}

    query = (
        f"Decision case {tid}. Multiple sources describe a changing operational situation. "
        f"The request is to determine whether available evidence is sufficient, which checks are necessary, "
        f"and what decision-support route should be followed. Report uncertainty explicitly and avoid unsupported
            certainty. "
        f"Case complexity marker {level}; evidence may contain omissions, stale observations, and conflicting
            interpretations."
    )

    n_records = int(round(5 + 18 * complexity + rng.integers(0, 5)))
    rel_rate = 0.30 + 0.28 * (1 - uncertainty)
    records = []
    vocab = [
        "sensor observation", "historical precedent", "operator note", "system status",
        "identity confidence", "time stamp", "resource state", "context update",
        "cross-source comparison", "previous correction", "technical constraint",
        "environmental condition", "uncertainty marker", "consistency check",
        "specialist note", "audit record"
    ]
    for j in range(n_records):
        relevant = bool(rng.random() < rel_rate)
        base = rng.choice(vocab)
        conf = rng.choice(["low", "medium", "high"])
        freshness = rng.choice(["current", "recent", "stale"])
        contradiction_phrase = "conflicts with another source" if (rng.random() < contradiction * 0.35) else "is
            internally consistent"
        filler_n = int(12 + 35 * complexity + rng.integers(0, 18))
        filler = " ".join(rng.choice(vocab, filler_n, replace=True))
        marker = "DECISION-RELEVANT" if relevant else "BACKGROUND"
        text = (f"Record {j+1} [{marker}]: {base}; confidence={conf}; temporal_status={freshness}; "
                f"this observation {contradiction_phrase}. {filler}.")
        records.append((text, relevant))

    return Task(tid, level, complexity, uncertainty, contradiction, risk, history_relevance,
                specialization_need, info_gap, seed, query, records, required, observed)


def train_tokenizer(tasks: List[Task], out_dir: Path, seed: int = 7) -> Path:
    out_dir.mkdir(parents=True, exist_ok=True)
    existing = out_dir / "qaf_bpe.model"
    if existing.exists():
        return existing
    corpus = out_dir / "corpus.txt"
    with corpus.open("w", encoding="utf-8") as f:
        for t in tasks:
            f.write(t.query + "\n")
            for rec, _ in t.records:
                f.write(rec + "\n")
        for x in SYSTEM_INSTRUCTIONS.values():
            f.write(x + "\n")
        # Include output vocabulary in training corpus.
        for a in AGENTS:
            f.write(f"Agent {a} completed structured analysis. Evidence assessed confidence uncertainty
                unresolved gaps recommendation escalation status.\n")
    model_prefix = str(out_dir / "qaf_bpe")
    spm.SentencePieceTrainer.train(
        input=str(corpus), model_prefix=model_prefix, vocab_size=1200,
```

```
        model_type="bpe", character_coverage=1.0,
        bos_id=-1, eos_id=-1, pad_id=-1, unk_id=0,
        shuffle_input_sentence=False, num_threads=1,
        train_extremely_large_corpus=False,
    )
    return out_dir / "qaf_bpe.model"


_TOKEN_CACHE = {}
def count_tokens(sp: spm.SentencePieceProcessor, text: str) -> int:
    v = _TOKEN_CACHE.get(text)
    if v is None:
        v = len(sp.encode(text, out_type=int))
        _TOKEN_CACHE[text] = v
    return v


def all_context(t: Task) -> str:
    return "\n".join(x for x, _ in t.records)


def targeted_context(t: Task, agent: str) -> str:
    rel = [x for x, r in t.records if r]
    bg = [x for x, r in t.records if not r]
    # Agent-specific budget: preserve all relevant evidence only for verification/retrieval;
    # other agents receive a smaller relevant slice and at most one background record.
    if agent in {"verification", "retrieval"}:
        chosen = rel[: max(1, min(len(rel), 10))]
    elif agent == "memory":
        chosen = rel[: max(1, min(len(rel), 6))]
    elif agent == "specialist":
        chosen = rel[: max(1, min(len(rel), 7))]
    elif agent == "planner":
        chosen = rel[: max(1, min(len(rel), 5))]
    else:
        chosen = rel[: max(1, min(len(rel), 4))]
    if not chosen and bg:
        chosen = bg[:1]
    return "\n".join(chosen)


def qaf_state_summary(t: Task) -> str:
    o = t.observed
    return (
        "QAF metacognitive state: "
        f"uncertainty={o['uncertainty']:.2f}; contradiction={o['contradiction']:.2f}; risk={o['risk']:.2f}; "
        f"history_relevance={o['history_relevance']:.2f}; specialization_need={o['specialization_need']:.2f}; "
        f"information_gap={o['info_gap']:.2f}; complexity={o['complexity']:.2f}."
    )


def route_rules(t: Task) -> Set[str]:
    o = t.observed
    r = {"reporter"}
    if o["history_relevance"] > 0.62: r.add("memory")
    if o["uncertainty"] > 0.62: r.add("retrieval")
    if o["specialization_need"] > 0.63: r.add("specialist")
    if o["contradiction"] > 0.56: r.add("verification")
    if o["complexity"] > 0.55: r.add("planner")
    return r


def route_qaf(t: Task, threshold: float = 0.50) -> Set[str]:
    """QAF-inspired metacognitive router.

    Scores use contextual cross-terms so uncertainty+information gap, contradiction+risk,
    and complexity+risk can activate a specialist route even when no single feature crosses
    a simple threshold.
    """
    o = t.observed
    scores = {
```

```
        "memory": 0.82*o["history_relevance"] + 0.18*o["complexity"],
        "retrieval": 0.55*o["uncertainty"] + 0.38*o["info_gap"] + 0.17*o["uncertainty"]*o["risk"],
        "specialist": 0.77*o["specialization_need"] + 0.17*o["complexity"] + 0.12*o["risk"],
        "verification": 0.58*o["contradiction"] + 0.32*o["risk"] + 0.24*o["contradiction"]*o["uncertainty"],
        "planner": 0.51*o["complexity"] + 0.34*o["risk"] + 0.18*o["uncertainty"]*o["complexity"],
    }
    r = {"reporter"}
    for a, s in scores.items():
        if s >= threshold:
            r.add(a)
    # Safety gate: high observed risk always gets verification+planning.
    if o["risk"] >= 0.82:
        r |= {"verification", "planner"}
    return r


def output_text(agent: str, t: Task, compact: bool) -> str:
    """Deterministic structured response emulator; tokenized like any other text.

    This is intentionally not an LLM. It represents the message volume an agent would
    contribute under the architecture. Compact QAF messages encode only sufficient state.
    """
    base = {
        "memory": "Prior evidence was checked against the current case. Relevant historical facts are separated
            from stale or unrelated records.",
        "retrieval": "Available evidence was inspected for missing facts. The response lists unresolved gaps and
            indicates whether additional evidence is necessary.",
        "specialist": "Technical constraints were evaluated. The analysis states the principal assumption, the
            most important limitation, and the recommended interpretation.",
        "verification": "Cross-source consistency was tested. Contradictions, low-confidence claims, and
            conditions requiring human review are explicitly identified.",
        "planner": "A reversible action sequence was prepared. The plan prioritizes information gain, preserves
            options under uncertainty, and defines escalation conditions.",
        "reporter": "Decision-support summary: evidence, uncertainty, confidence, unresolved issues, rationale,
            and escalation status are consolidated for the operator.",
    }[agent]
    repetitions = max(1, int(round((1.0 + 2.2*t.complexity) * (0.62 if compact else 1.0))))
    details = []
    for j in range(repetitions):
        details.append(
            f"Assessment segment {j+1}: uncertainty {t.observed['uncertainty']:.2f}, "
            f"risk {t.observed['risk']:.2f}, contradiction {t.observed['contradiction']:.2f}; "
            "evidence is prioritized by relevance and confidence."
        )
    return base + " " + " ".join(details)


def build_call(t: Task, agent: str, arch: str) -> Tuple[str, str]:
    compact = arch in {"QAFRouter", "Oracle"}
    ctx = targeted_context(t, agent) if compact else all_context(t)
    state = ("\n" + qaf_state_summary(t)) if arch == "QAFRouter" else ""
    prompt = (
        f"SYSTEM: {SYSTEM_INSTRUCTIONS[agent]}\n"
        f"USER: {t.query}\n"
        f"CONTEXT:\n{ctx}{state}\n"
        "Return a structured, decision-relevant response only."
    )
    return prompt, output_text(agent, t, compact=compact)


def evaluate_task(t: Task, arch: str, sp: spm.SentencePieceProcessor, threshold: float = 0.50) -> Dict:
    if arch == "AllAgents":
        selected = set(AGENTS)
    elif arch == "RulesRouter":
        selected = route_rules(t)
    elif arch == "QAFRouter":
        selected = route_qaf(t, threshold)
    elif arch == "Oracle":
        selected = set(t.required)
    else:
        raise ValueError(arch)
```

```
    # Component-wise BPE accounting. Counting components separately avoids repeatedly
    # tokenizing giant concatenated prompts; every architecture uses the same convention.
    scaffold_tokens = count_tokens(sp, "SYSTEM USER CONTEXT Return a structured, decision-relevant response only.
        ")
    query_tokens = count_tokens(sp, t.query)
    all_record_tokens = [count_tokens(sp, x) for x,_ in t.records]
    relevant_idx = [i for i,(_,r) in enumerate(t.records) if r]
    bg_idx = [i for i,(_,r) in enumerate(t.records) if not r]

    def context_token_count(agent):
        if arch not in {"QAFRouter", "Oracle"}:
            return sum(all_record_tokens)
        if agent in {"verification", "retrieval"}: cap=10
        elif agent == "memory": cap=6
        elif agent == "specialist": cap=7
        elif agent == "planner": cap=5
        else: cap=4
        ids = relevant_idx[:max(1,min(len(relevant_idx),cap))]
        if not ids and bg_idx:
            ids = bg_idx[:1]
        return sum(all_record_tokens[i] for i in ids)

    input_tokens = output_tokens = 0
    state_tokens = count_tokens(sp, qaf_state_summary(t)) if arch == "QAFRouter" else 0
    for agent in sorted(selected):
        input_tokens += scaffold_tokens + count_tokens(sp, SYSTEM_INSTRUCTIONS[agent]) + query_tokens +
            context_token_count(agent) + state_tokens
        output_tokens += count_tokens(sp, output_text(agent, t, compact=(arch in {"QAFRouter","Oracle"})))

    tp = len(selected & t.required)
    fp = len(selected - t.required)
    fn = len(t.required - selected)
    recall = tp / len(t.required) if t.required else 1.0
    precision = tp / len(selected) if selected else 1.0
    success = 1 if fn == 0 else 0
    quality = max(0.0, min(1.0, 0.82*recall + 0.18*precision))
    total = input_tokens + output_tokens
    return {
        "task_id": t.tid, "level": t.level, "architecture": arch,
        "complexity": t.complexity, "uncertainty": t.uncertainty,
        "contradiction": t.contradiction, "risk": t.risk,
        "required_agents": len(t.required), "selected_agents": len(selected),
        "input_tokens": input_tokens, "output_tokens": output_tokens, "total_tokens": total,
        "routing_recall": recall, "routing_precision": precision,
        "success": success, "quality": quality,
        "tokens_per_quality": total / max(quality, 1e-6),
        "selected_set": ",".join(sorted(selected)),
        "required_set": ",".join(sorted(t.required)),
    }


def make_figures(df: pd.DataFrame, sweep: pd.DataFrame, activation: pd.DataFrame, fig_dir: Path):
    fig_dir.mkdir(parents=True, exist_ok=True)
    order = ["AllAgents", "RulesRouter", "QAFRouter", "Oracle"]
    pretty = {"AllAgents":"All agents", "RulesRouter":"Rules router", "QAFRouter":"QAF router", "Oracle":"Oracle
        lower bound"}

    # 1 architecture diagram
    fig, ax = plt.subplots(figsize=(10,5.4)); ax.axis('off')
    ax.text(0.08,0.75,"Conventional",fontsize=14,weight='bold')
    ax.text(0.08,0.25,"QAF-inspired",fontsize=14,weight='bold')
    for y in [0.72,0.22]:
        ax.annotate("", xy=(0.94,y), xytext=(0.16,y), arrowprops=dict(arrowstyle="->",lw=1.5))
    xs=[0.18,0.38,0.60,0.80]
    labs_top=["Query + full context","All agents","All messages","Final synthesis"]
    labs_bot=["Query + state","Metacognitive routing","Targeted agents/context","Final synthesis"]
    for x,l1,l2 in zip(xs,labs_top,labs_bot):
        ax.text(x,0.72,l1,ha='center',va='center',bbox=dict(boxstyle='round,pad=.35',fc='white'))
        ax.text(x,0.22,l2,ha='center',va='center',bbox=dict(boxstyle='round,pad=.35',fc='white'))
    ax.set_title("Figure 1. Token-flow architectures compared",fontsize=16)
```

```
fig.tight_layout(); fig.savefig(fig_dir/'fig01_architecture.png',dpi=180); plt.close(fig)

# summary helpers
summ=df.groupby('architecture').agg(total_tokens=('total_tokens','mean'),input_tokens=('input_tokens','mean')
    ,output_tokens=('output_tokens','mean'),calls=('selected_agents','mean'),success=('success','mean'),
    quality=('quality','mean')).reindex(order)

# 2 total tokens
fig,ax=plt.subplots(figsize=(8,5)); vals=summ.total_tokens.values
ax.bar([pretty[x] for x in order], vals); ax.set_ylabel('Mean tokens per task'); ax.set_title('Figure 2.
    Total token consumption'); ax.tick_params(axis='x',rotation=18)
for i,v in enumerate(vals): ax.text(i,v,f'{v:,.0f}',ha='center',va='bottom')
fig.tight_layout(); fig.savefig(fig_dir/'fig02_total_tokens.png',dpi=180); plt.close(fig)

# 3 input/output stacked
fig,ax=plt.subplots(figsize=(8,5)); x=np.arange(len(order)); inp=summ.input_tokens.values; out=summ.
    output_tokens.values
ax.bar(x,inp,label='Input'); ax.bar(x,out,bottom=inp,label='Output'); ax.set_xticks(x,[pretty[z] for z in
    order],rotation=18); ax.set_ylabel('Mean tokens per task'); ax.set_title('Figure 3. Input vs output
    token footprint'); ax.legend()
fig.tight_layout(); fig.savefig(fig_dir/'fig03_input_output.png',dpi=180); plt.close(fig)

# 4 reduction by complexity vs all agents
lvl_order=['low','medium','high']
p=df.groupby(['level','architecture']).total_tokens.mean().unstack()
reduction=(1-p.div(p['AllAgents'],axis=0))*100
fig,ax=plt.subplots(figsize=(8,5))
for a in ['RulesRouter','QAFRouter','Oracle']:
    ax.plot(lvl_order,reduction.loc[lvl_order,a],marker='o',label=pretty[a])
ax.set_ylabel('Token reduction vs AllAgents (%)'); ax.set_xlabel('Task complexity'); ax.set_title('Figure 4.
     Token savings as complexity rises'); ax.legend(); ax.set_ylim(bottom=0)
fig.tight_layout(); fig.savefig(fig_dir/'fig04_reduction_complexity.png',dpi=180); plt.close(fig)

# 5 calls per task
fig,ax=plt.subplots(figsize=(8,5)); vals=summ.calls.values
ax.bar([pretty[x] for x in order],vals); ax.set_ylabel('Mean agent calls'); ax.set_title('Figure 5.
    Communication calls per task'); ax.tick_params(axis='x',rotation=18)
for i,v in enumerate(vals): ax.text(i,v,f'{v:.2f}',ha='center',va='bottom')
fig.tight_layout(); fig.savefig(fig_dir/'fig05_calls.png',dpi=180); plt.close(fig)

# 6 Pareto cost-quality
fig,ax=plt.subplots(figsize=(7,5));
for a in order:
    ax.scatter(summ.loc[a,'total_tokens'],summ.loc[a,'quality']*100,s=90,label=pretty[a]); ax.annotate(pretty
        [a],(summ.loc[a,'total_tokens'],summ.loc[a,'quality']*100),xytext=(5,5),textcoords='offset points',
        fontsize=8)
ax.set_xlabel('Mean tokens per task'); ax.set_ylabel('Mean routing quality (%)'); ax.set_title('Figure 6.
    Token-quality frontier'); ax.grid(alpha=.25)
fig.tight_layout(); fig.savefig(fig_dir/'fig06_pareto.png',dpi=180); plt.close(fig)

# 7 tokens per successful task
tmp=df.copy(); tmp['success_tokens']=np.where(tmp.success==1,tmp.total_tokens,np.nan)
successful_mean=tmp.groupby('architecture').success_tokens.mean().reindex(order)
fig,ax=plt.subplots(figsize=(8,5)); vals=successful_mean.values
ax.bar([pretty[x] for x in order],vals); ax.set_ylabel('Tokens, successful tasks only'); ax.set_title('
    Figure 7. Token cost of a successful route'); ax.tick_params(axis='x',rotation=18)
fig.tight_layout(); fig.savefig(fig_dir/'fig07_tokens_success.png',dpi=180); plt.close(fig)

# 8 distribution p50 p95
stats=df.groupby('architecture').total_tokens.quantile([.5,.95]).unstack().reindex(order)
fig,ax=plt.subplots(figsize=(8,5)); x=np.arange(len(order)); w=.36
ax.bar(x-w/2,stats[0.5],w,label='p50'); ax.bar(x+w/2,stats[0.95],w,label='p95'); ax.set_xticks(x,[pretty[z]
    for z in order],rotation=18); ax.set_ylabel('Tokens per task'); ax.set_title('Figure 8. Median and tail
    token consumption'); ax.legend()
fig.tight_layout(); fig.savefig(fig_dir/'fig08_p50_p95.png',dpi=180); plt.close(fig)

# 9 activation heatmap for QAF
q=activation[activation.architecture=='QAFRouter'].pivot(index='agent',columns='level',values='
    activation_rate').reindex(AGENTS)[lvl_order]
fig,ax=plt.subplots(figsize=(7.5,5.2)); im=ax.imshow(q.values,aspect='auto',vmin=0,vmax=1)
ax.set_xticks(range(3),lvl_order); ax.set_yticks(range(len(AGENTS)),AGENTS); ax.set_title('Figure 9. QAF
```

```
        agent activation by complexity')
    for i in range(q.shape[0]):
        for j in range(q.shape[1]): ax.text(j,i,f'{q.iloc[i,j]*100:.0f}%',ha='center',va='center')
    fig.colorbar(im,ax=ax,label='Activation rate'); fig.tight_layout(); fig.savefig(fig_dir/'
        fig09_activation_heatmap.png',dpi=180); plt.close(fig)

    # 10 threshold sensitivity
    fig,ax1=plt.subplots(figsize=(8,5)); ax2=ax1.twinx()
    ax1.plot(sweep.threshold,sweep.mean_tokens,marker='o',label='Tokens'); ax2.plot(sweep.threshold,sweep.
        success_rate*100,marker='s',linestyle='--',label='Success')
    ax1.set_xlabel('QAF routing threshold'); ax1.set_ylabel('Mean tokens'); ax2.set_ylabel('Routing success (%)'
        ); ax1.set_title('Figure 10. QAF threshold: efficiency-quality trade-off')
    fig.tight_layout(); fig.savefig(fig_dir/'fig10_threshold_sensitivity.png',dpi=180); plt.close(fig)


def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--out',default='/mnt/data/qaf_tokens'); ap.add_argument('--n-
        per-level',type=int,default=1000); ap.add_argument('--seed',type=int,default=20260919)
    args=ap.parse_args(); out=Path(args.out); (out/'results').mkdir(parents=True,exist_ok=True); (out/'figures').
        mkdir(exist_ok=True); (out/'tokenizer').mkdir(exist_ok=True)
    tasks=[]; tid=0
    for li,level in enumerate(['low','medium','high']):
        for i in range(args.n_per_level):
            tid+=1; tasks.append(generate_task(tid,level,args.seed+li*100000+i))
    model=train_tokenizer(tasks,out/'tokenizer')
    sp=spm.SentencePieceProcessor(model_file=str(model))

    rows=[]
    for t in tasks:
        for arch in ['AllAgents','RulesRouter','QAFRouter','Oracle']:
            rows.append(evaluate_task(t,arch,sp,threshold=.35))
    df=pd.DataFrame(rows); df.to_csv(out/'results'/'raw_results.csv',index=False)

    summary=df.groupby('architecture').agg(
        tasks=('task_id','count'),mean_input_tokens=('input_tokens','mean'),mean_output_tokens=('output_tokens','
            mean'),mean_total_tokens=('total_tokens','mean'),
        median_tokens=('total_tokens','median'),p95_tokens=('total_tokens',lambda x: np.quantile(x,.95)),
            mean_agent_calls=('selected_agents','mean'),
        routing_recall=('routing_recall','mean'),routing_precision=('routing_precision','mean'),success_rate=('
            success','mean'),mean_quality=('quality','mean')
    ).reset_index()
    base=float(summary.loc[summary.architecture=='AllAgents','mean_total_tokens'].iloc[0])
    summary['token_reduction_vs_all_pct']=(1-summary.mean_total_tokens/base)*100
    summary['tokens_per_success_equivalent']=summary.mean_total_tokens/summary.success_rate.replace(0,np.nan)
    summary.to_csv(out/'results'/'summary.csv',index=False)

    # threshold sweep on a 900-task fixed subset for efficiency.
    subset=tasks[::max(1,len(tasks)//900)][:900]
    sw=[]
    for th in np.arange(.35,.76,.05):
        rr=[evaluate_task(t,'QAFRouter',sp,threshold=float(th)) for t in subset]
        d=pd.DataFrame(rr)
        sw.append(dict(threshold=round(float(th),2),mean_tokens=d.total_tokens.mean(),success_rate=d.success.mean
            (),quality=d.quality.mean(),mean_calls=d.selected_agents.mean()))
    sweep=pd.DataFrame(sw); sweep.to_csv(out/'results'/'threshold_sweep.csv',index=False)

    # Agent activation rates by architecture/complexity.
    ar=[]
    for arch in ['RulesRouter','QAFRouter']:
        sub=df[df.architecture==arch]
        for level in ['low','medium','high']:
            ss=sub[sub.level==level]
            for agent in AGENTS:
                ar.append(dict(architecture=arch,level=level,agent=agent,activation_rate=ss.selected_set.str.
                    contains(fr'(^|,){agent}(,|$)',regex=True).mean()))
    activation=pd.DataFrame(ar); activation.to_csv(out/'results'/'agent_activation.csv',index=False)

    make_figures(df,sweep,activation,out/'figures')

    metadata={
        'n_tasks':len(tasks),'n_architecture_evaluations':len(df),'seed':args.seed,'tokenizer':'SentencePiece BPE
```

```
            , 1200 vocabulary, benchmark-trained',
        'scientific_scope':'architecture-level token accounting; no external LLM inference',
    }
    (out/'results'/'metadata.json').write_text(json.dumps(metadata,indent=2),encoding='utf-8')
    print(summary.to_string(index=False))
    print('\nQAF by level:')
    print(df[df.architecture=='QAFRouter'].groupby('level').agg(tokens=('total_tokens','mean'),success=('success'
        ,'mean'),quality=('quality','mean'),calls=('selected_agents','mean')).to_string())

if __name__=='__main__': main()
```